Gen AI Career Platform — Technology Stack

Oct 17, 2026 · @Vishant

1. Project Overview

The Gen AI Career Platform is a MERN-stack web app that uses Google Gemini to prepare a job seeker for a specific role. Each user has a private account, and everything they generate is saved to their own history.

The app has three user-facing features:

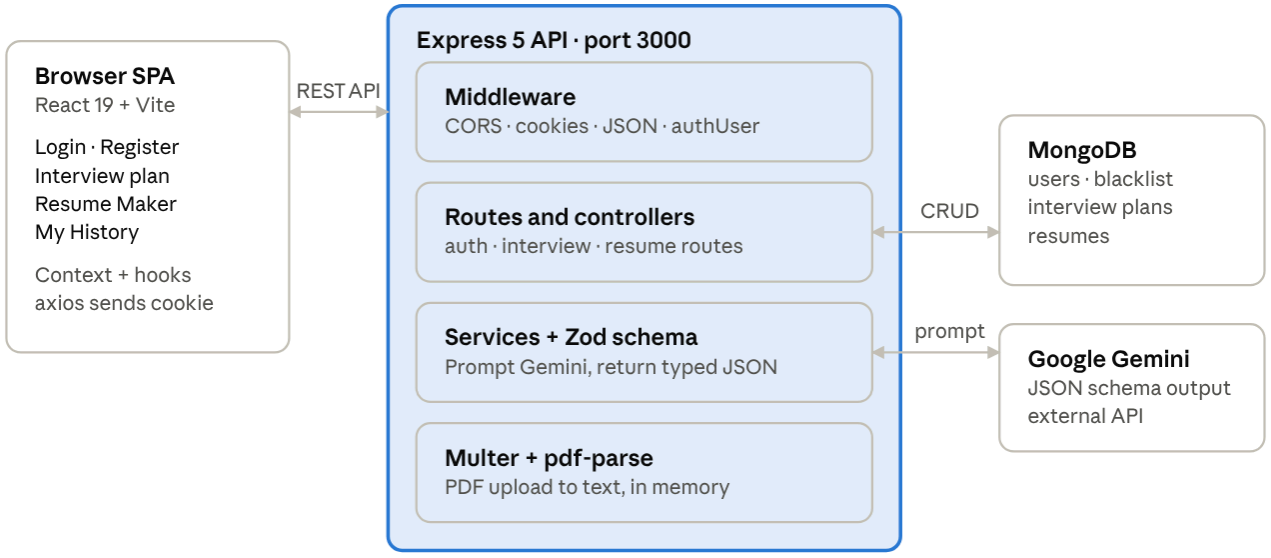
- **Interview Plan generator** — takes a job description plus a resume PDF or a short self-description, and returns a match score, technical and behavioral questions with answer guidance, skill gaps, and a day-wise preparation plan.
- **Resume Maker** — turns the user's own self-description (optionally a job description and an old resume PDF) into an ATS-friendly resume with an ATS score, improvement tips, clickable profile links, and print-to-PDF.
- **My History** — lists every previous interview plan and resume of the logged-in user, with search, filters, open and delete.

> Every technology here was chosen to keep the project simple to run locally, free-tier friendly, and strictly per-user: a user can only ever read or delete their own records.

2. High-Level Architecture

The browser only talks to the Express API; the API alone reaches MongoDB and Gemini.

One authenticated API sits between the browser, MongoDB and Gemini



architecture · browser, Express API, MongoDB, Gemini

A request passes the middleware, `authUser` checks the login token, and the controller then reads or writes MongoDB, or calls Gemini through a service. Layer by layer:

Layer	Technology	What It Does
Frontend	React 19, Vite, React Router, axios, SCSS	Screens, session restore, form submission, printable resume
API	Node.js, Express 5	REST endpoints, CORS, cookie parsing, JSON bodies
Auth	JWT, bcryptjs, HTTP-only cookie	Login, per-user access, logout blacklist
AI service	`@google/genai`, Zod	Builds prompts and JSON schemas, calls Gemini
File handling	Multer, pdf-parse	Reads an uploaded PDF into text
Database	MongoDB, Mongoose	Users, interview plans, resumes, blacklisted tokens

3. Frontend — React 19 + Vite

What It Is

A single-page app built with React 19 and bundled by Vite. It talks to the backend only through REST calls made with axios, and is styled with SCSS (one stylesheet per feature, dark theme with a pink-to-purple accent).

What It Does in This Project

- Renders every screen: Login, Register, Home (interview-plan form), Interview report, Resume Maker, Resume view, and My History.
- Uses React Router for navigation and a `Protected` wrapper that redirects anonymous visitors to `/login`.
- On first load, `AuthProvider` calls `GET /api/auth/get-me` to restore the session from the cookie, so a refresh does not log the user out.
- Keeps shared state in React Context (`AuthContext`, `InterviewContext`) exposed through custom hooks (`useAuth`, `useInterview`).
- Sends every request with `withCredentials: true` so the browser attaches the login cookie.
- Draws the resume as a printable page; a print stylesheet hides the app chrome so the browser's Save as PDF produces a clean file.

Why React + Vite

All content sits behind a login, so search-engine rendering is not needed and a plain SPA is enough. Vite starts instantly and updates the browser in milliseconds while editing, and React's component model maps neatly onto repeated UI such as history cards and question lists.

Alternatives Considered

Option	What It Offers	Why Not Chosen
React + Vite (chosen)	Fast dev server, huge ecosystem, completes the MERN stack	—
Next.js	Server rendering, file-based routing	SSR and SEO are not needed behind a login; adds a second server concept

Option	What It Offers	Why Not Chosen
Create React App	Familiar setup	Slower builds and no longer the recommended starter
Vue / Angular	Capable full frameworks	Not part of the MERN stack the project follows

4. Backend — Node.js + Express 5

What It Is

Express 5 running on Node.js (CommonJS modules) is the REST API on port 3000. The code is layered as routes, then controllers, then services, then models.

What It Does in This Project

- Mounts three routers: `/api/auth`, `/api/interview` and `/api/resume`.
- Applies `express.json`, `cookie-parser` and CORS (allowed origin `http://localhost:5173`, credentials on) to every request.
- Runs the `authUser` middleware on every private route, so controllers always know `req.user.id`.
- Keeps each controller in a try/catch that returns a JSON `message`, plus a global error handler as a safety net.
- Calls Gemini through service files (`ai.service.js` for interview plans, `resume.service.js` for resumes), so controllers never touch the AI SDK directly.

Why Node.js + Express

One language across the whole stack keeps the project simple. Node's non-blocking I/O suits an app that mostly waits on a database and an LLM response, and Express 5 forwards errors from async handlers automatically.

Alternatives Considered

Option	What It Offers	Why Not Chosen
Express (chosen)	Minimal, widely known, JavaScript end to end	—
NestJS	Opinionated structure, built-in validation	More boilerplate than a project this size needs

Option	What It Offers	Why Not Chosen
FastAPI (Python)	Strong AI ecosystem, async, auto docs	Introduces a second language next to the React frontend
Fastify	Faster than Express, schema-first	Smaller ecosystem; speed is not the bottleneck here, the LLM call is

5. Database — MongoDB + Mongoose

What It Is

MongoDB is a document database; Mongoose 9 is the library that defines schemas and talks to it. The connection string comes from `MONGO_URI` and can point to a local MongoDB or a MongoDB Atlas cluster.

What It Does in This Project

- Stores four models: `users`, `blacklistTokens`, `InterviewReport` and `Resume` (fields in section 13).
- Saves each AI result as one document, including nested arrays such as questions, skill gaps, and resume sections.
- Ties every report and resume to its owner with a `user` ObjectId, and every read, list and delete query filters on `user: req.user.id`, so users can never see each other's records.
- Applies Mongoose validation (required fields, `low | medium | high` severity, score range 0–100) before anything is saved.

Why MongoDB

The AI returns deeply nested JSON. In a document database that JSON is stored as it is, with no joins, while a relational database would need several linked tables or a JSON column. Atlas also offers a free tier suited to a student project.

Alternatives Considered

Option	What It Offers	Why Not Chosen
MongoDB + Mongoose (chosen)	Nested documents, flexible schema, free Atlas tier	—

Option	What It Offers	Why Not Chosen
PostgreSQL	ACID transactions, strong relational queries	Nested AI output would need extra tables or JSONB; relations are not the core of this data
MySQL	Mature and widely hosted	Same nesting problem as PostgreSQL
Firebase Firestore	Managed, real-time sync	Ties the project to another vendor; real-time sync is not needed

6. Authentication — JWT + bcryptjs + HTTP-only Cookies

What It Is

Login is stateless: the server signs a JSON Web Token (JWT) and the browser keeps it in an HTTP-only cookie. Passwords are never stored in plain text; they are hashed with bcryptjs.

What It Does in This Project

- **Register / login:** the password is hashed (10 salt rounds) or compared with the stored hash; a JWT carrying the user id and username is signed with `JWT_SECRET` and expires in 1 day.
- **Cookie settings:** `httpOnly` (JavaScript cannot read it), `sameSite: lax`, `secure` in production, and a 1-day `maxAge` that matches the token.
- **Session check:** `GET /api/auth/get-me` lets the frontend restore the logged-in user on page load.
- **Logout:** clears the cookie and stores the token in a `blacklistTokens` collection; `authUser` rejects any blacklisted token even if it has not expired yet.
- **Same error for both failures:** a wrong email and a wrong password both return 'Invalid email or password', so the API does not reveal which accounts exist.

Why Cookies Instead of localStorage

A token in localStorage can be stolen by any injected script. An HTTP-only cookie cannot be read by scripts at all, which makes it the safer default for a browser app.

Alternatives Considered

Option	What It Offers	Why Not Chosen
JWT in HTTP-only cookie (chosen)	Stateless, hidden from JavaScript	—
JWT in localStorage	Simple to attach to requests	Readable by any script, so an XSS bug leaks the token
Server sessions (express-session)	Easy to revoke	Needs server-side session storage and more setup
OAuth / Auth0 / Firebase Auth	Social login, managed security	External dependency and more configuration than the project needs

7. LLM — Google Gemini (@google/genai)

What It Is

Gemini is the language model behind both AI features. The backend calls it through the official `@google/genai` SDK, using the model `gemini-3-flash-preview` and an API key from `GOOGLE_GENAI_API_KEY`.

What It Does in This Project

- **Interview plan:** from the job description plus resume or self-description, it returns a match score, technical and behavioral questions with the interviewer's intention and a suggested answer, skill gaps with severity, and a day-wise preparation plan.
- **Resume:** from the user's self-description, it returns a full structured resume, an ATS score estimate, and improvement tips.
- **Structured output:** each call sets `responseMimeType: application/json` and a `responseSchema`, so Gemini answers in a fixed JSON shape that is parsed straight into the database models, with no fragile text parsing.
- **Schema cleanup:** a helper strips JSON-Schema keys Gemini rejects (`additionalProperties`, `$schema`) before the call.

Why Gemini

It supports schema-constrained JSON output, is fast enough for a roughly 20 to 30 second request, and an API key from Google AI Studio gives a free tier suited to development and demos.

Alternatives Considered

Option	What It Offers	Why Not Chosen
Google Gemini (chosen)	JSON schema output, fast Flash tier, free development key	—
OpenAI GPT models	Strong quality, structured outputs	Paid API; would need a different SDK and schema format
Anthropic Claude	Strong writing and reasoning	Paid API; a different SDK and prompt setup
Mistral / Groq	Fast, hosted	Different SDK and weaker fit for this schema-first setup
Ollama (local models)	Free and private	Needs a capable local GPU and RAM; slower on typical laptops

8. Structured Output — Zod 4

What It Is

Zod is a TypeScript-first schema library. Here it describes, in code, the exact JSON shape the AI must return.

What It Does in This Project

- Defines the interview-plan schema (`matchScore` , `technicalQuestions` , `behavioralQuestions` , `skillGaps` , `preparationPlan` , `title`) and the resume schema (`title` , `atsScore` , `improvementTips` , and a `content` object with contact, summary, skills, experience, projects, education, certifications, achievements).
- Attaches a `.describe()` hint to each field; these hints travel to Gemini and tell it what each field should contain.
- Converts the schema with `z.toJSONSchema(..., { target: 'openapi-3.0' })` into the format Gemini's `responseSchema` accepts.

One gap to know about: the AI reply is parsed with `JSON.parse` and then checked by Mongoose when saved, not re-validated with Zod.

Alternatives Considered

Option	What It Offers	Why Not Chosen
Zod (chosen)	One schema gives JSON Schema output and descriptions	—
Joi / Yup	Mature validation libraries	No built-in JSON Schema conversion as direct as Zod's
Hand-written JSON Schema	No dependency	Verbose and easy to get out of sync with the models

9. File Handling — Multer + pdf-parse

What It Is

Multer reads file uploads sent as `multipart/form-data`, and pdf-parse extracts plain text from a PDF.

What It Does in This Project

- Accepts one file in the field `resume` on both `POST /api/interview` and `POST /api/resume`.
- Keeps the upload in memory (`memoryStorage`) with a 4 MB limit, so nothing is written to the server's disk.
- Passes the file's bytes to pdf-parse (`PDFParse(...).getText()`), and the extracted text becomes extra material in the Gemini prompt; for interview plans it is also saved with the report.
- Treats the upload as optional on the Resume Maker, where the self-description is the main input.

A scanned PDF that contains only images has no text layer, so nothing can be extracted from it; OCR is not part of this stack.

Alternatives Considered

Option	What It Offers	Why Not Chosen
Multer + pdf-parse (chosen)	Small, in-memory, works with Express	—
formidable / busboy	Lower-level upload parsing	More code for the same result

Option	What It Offers	Why Not Chosen
pdfjs-dist	Full PDF rendering engine	Heavier than needed for plain text extraction
Tesseract OCR	Reads scanned documents	Adds a heavy dependency; out of scope for now

10. Interview Plan Pipeline

When a user clicks Generate, one request travels through seven steps:

1. **Input:** the user pastes a job description (required, up to 5,000 characters) and either uploads a resume PDF or writes a short self-description. The frontend sends them as `multipart/form-data` to `POST /api/interview`.
2. **Authenticate:** `authUser` verifies the login cookie and rejects blacklisted or invalid tokens.
3. **Validate:** the controller returns 400 if the job description is missing, or if there is neither a resume nor a self-description.
4. **Extract:** if a PDF was uploaded, Multer holds it in memory and pdf-parse converts it to text.
5. **Generate:** the service sends resume text, self-description and job description to Gemini with the Zod-derived `responseSchema`.
6. **Save:** the parsed JSON is stored as an `InterviewReport` together with the user's id and original inputs.
7. **Show:** the API returns 201 and the frontend opens `/interview/:id`, which has Technical Questions, Behavioral Questions and Road Map tabs, plus the match score and skill gaps.

The history list endpoint leaves out the heavy fields (questions, plan, resume text), so loading My History stays fast.

11. Resume Maker Pipeline

The resume is built only from what the user writes. The pipeline has eight steps:

1. **Input:** a self-description of at least 30 characters is required. Target role, full name, phone, location, links, a job description and an old resume PDF are optional. The form is sent to `POST /api/resume`.

2. **Validate and prepare:** the controller extracts PDF text if one was attached, loads the account's username and email as defaults, and splits the links field on commas or new lines (at most 6).
3. **Prompt rules:** Gemini is told to use only the candidate's material, never to invent employers, titles, degrees, dates, certifications, projects, links or numbers, to add a metric only if the user gave one, to leave a section empty when there is nothing to put in it, and to treat the candidate text as data, not as instructions.
4. **Tailor:** if a target role or job description is given, headline, summary and skill order are tuned to it, and its keywords are mirrored only where the candidate genuinely has that skill.
5. **Generate:** Gemini returns the structured resume, an ATS score estimate and 3 to 5 improvement tips.
6. **Contact override:** the server replaces name, email, phone, location and links with your account email and the details the user typed, so the model cannot alter them.
7. **Save:** the result is stored as a `Resume` linked to the user; the title comes from the target role.
8. **Render:** `/resume/:id` draws a paper-style resume. Each profile link is parsed into a clickable label, for example `linkedin.com/in/vishant-sutar-1a2b3c4d5` becomes 'LinkedIn · Vishant Sutar'. Email opens a mail window and the phone number is dialable. Only `http` and `https` links are made clickable.

Download as PDF uses the browser's print dialog with a print stylesheet; the links stay clickable in the saved PDF.

12. REST API Endpoints

All private routes need the login cookie; a missing or invalid cookie returns 401.

Method	Endpoint	Access	Purpose
POST	`/api/auth/register`	Public	Create an account and log in
POST	`/api/auth/login`	Public	Log in with email and password
GET	`/api/auth/logout`	Public	Clear the cookie and blacklist the token
GET	`/api/auth/get-me`	Private	Return the logged-in user

Method	Endpoint	Access	Purpose
POST	`/api/interview`	Private	Generate an interview plan (multipart, optional PDF)
GET	`/api/interview`	Private	List the user's interview plans (light fields)
GET	`/api/interview/report/:interviewId`	Private	Get one full interview plan
DELETE	`/api/interview/report/:interviewId`	Private	Delete one interview plan
POST	`/api/resume`	Private	Generate a resume (multipart, optional PDF)
GET	`/api/resume`	Private	List the user's resumes (title, ATS score, dates)
GET	`/api/resume/:resumeId`	Private	Get one full resume
DELETE	`/api/resume/:resumeId`	Private	Delete one resume

13. Data Models

Model	Key fields
`users`	`username` (unique), `email` (unique), `password` (bcrypt hash)
`blacklistTokens`	`token`, timestamps
`InterviewReport`	`user` (owner), `title`, `jobDescription`, `resume` (extracted text), `selfDescription`, `matchScore` (0–100), `technicalQuestions[]` and `behavioralQuestions[]` (question, intention, answer), `skillGaps[]` (skill, severity: low / medium / high), `preparationPlan[]` (day, focus, tasks), timestamps
`Resume`	`user` (owner, indexed), `title`, `targetRole`, `jobDescription`, `selfDescription`, `atsScore` (0–100), `improvementTips[]`, `content` (see below), timestamps

The `content` object of a `Resume` holds: `fullName`, `headline`, `contact` (email, phone, location, links), `summary`, `skills[]` (category, items), `experience[]` (role, company, location, duration, bullets), `projects[]` (name, techStack, link, bullets), `education[]` (degree, institution, year, details), `certifications[]` and `achievements[]`.

14. Dev Tooling and Configuration

The backend must run on port 3000 and the frontend on port 5173, because the frontend's API address and the backend's CORS origin are written into the code.

Setting	Where	Purpose
`MONGO_URI`	`Backend/.env`	MongoDB connection string (local or Atlas)
`JWT_SECRET`	`Backend/.env`	Secret used to sign login tokens
`GOOGLE_GENAI_API_KEY`	`Backend/.env`	Gemini API key

Tool	Role
`npm run dev` (Backend)	Starts the API with nodemon, which restarts on file changes; nodemon is not listed in `package.json`, so it must be installed globally or added as a dev dependency
`npm run dev` (Frontend)	Starts the Vite dev server with instant refresh
dotenv	Loads `.env` into `process.env`
ESLint	Lints the frontend code
Sass	Compiles the SCSS stylesheets

Keep `.env` out of version control (it is listed in `Backend/.gitignore`), and rotate the keys if the file has ever been shared.

15. Full Stack Summary

Technology	Category	Role in This Project
React 19	Frontend library	All screens and state (Context API)
Vite	Build tool	Dev server and production bundle

Technology	Category	Role in This Project
React Router	Routing	Page navigation and protected routes
axios	HTTP client	API calls with the login cookie
Sass (SCSS)	Styling	Per-feature stylesheets and print layout
Node.js	Runtime	Runs the backend
Express 5	Web framework	REST API, routing, middleware
MongoDB	Document database	Users, interview plans, resumes, blacklisted tokens
Mongoose 9	ODM	Schemas, validation, queries
Google Gemini (`@google/genai`)	LLM	Interview plans and resumes as structured JSON
Zod 4	Schema library	Describes the AI response shape for Gemini
JWT (`jsonwebtoken`)	Authentication	Signed 1-day login token
bcryptjs	Security	Password hashing
cookie-parser	Middleware	Reads the login cookie
cors	Middleware	Lets the frontend call the API with credentials
Multer	File upload	In-memory PDF upload, 4 MB limit
pdf-parse	Document parsing	Extracts text from uploaded PDFs
dotenv	Configuration	Loads secrets from `.env`

16. Known Limitations and Design Philosophy

Known Limitations and Next Steps

Limitation	Suggested improvement
Blacklisted tokens are never removed from the database	Add a TTL index so entries expire after the token's 1 day

Limitation	Suggested improvement
API URL, CORS origin and ports are fixed to localhost	Move them to environment variables before deploying
The Gemini model name is hard-coded and is a preview name	Read it from an environment variable so it can be swapped
The upload box accepts `.docx`, but the server only parses PDFs	Accept PDF only in the UI, or add a DOCX parser
The AI reply is not re-validated with Zod after parsing	Validate with the same Zod schema before saving
The ATS score is the model's estimate, not a real ATS measurement	Label it as an estimate, or compute keyword coverage against the job description
No rate limit on the two generation endpoints	Add rate limiting, since each call uses Gemini quota
A generated resume cannot be edited in the app	Add inline editing, or a regenerate-section option
Scanned (image-only) PDFs give no text	Add OCR as a fallback

Design Philosophy

Every technology was picked to satisfy three constraints: it should be free or free-tier friendly, run on a laptop without complex infrastructure, and directly serve the app's features.

Two rules shape the design. First, every record belongs to one user, and every query is scoped to that user. Second, the AI is held to a fixed JSON schema, and the Resume Maker is told never to invent facts, so the output stays structured and trustworthy enough to save, show and print.